



Tema 4

Diseño orientado a objetos

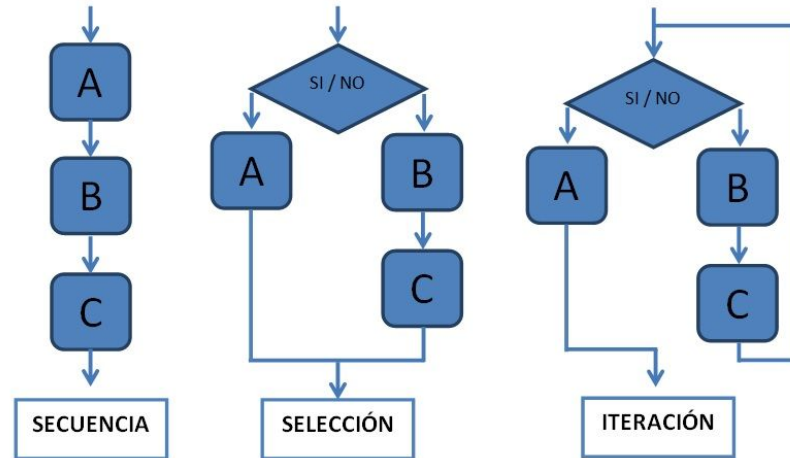
Introducción

El desarrollo de software es un proceso cuyo objetivo es dar solución a problemas utilizando una herramienta informática y tiene como resultado la construcción de un programa informático. Como en cualquier otra disciplina en la que se obtenga un producto final de cierta complejidad, si queremos obtener un producto de calidad, es preciso realizar un proceso previo de análisis y especificación del proceso que vamos a seguir, y de los resultados que pretendemos conseguir.



Programación estructurada

Sin embargo, cómo se hace es algo que ha ido evolucionando con el tiempo, en un principio se tomaba el problema de partida y se iba sometiendo a un proceso de división en subproblemas más pequeños reiteradas veces, hasta que se llegaba a problemas elementales que se podía resolver utilizando una función. Luego las funciones se hilaban hasta formar una solución global al problema de partida. Era, pues, un proceso centrado en los procedimientos, se codificaban mediante funciones que actuaban sobre estructuras de datos, por eso a este tipo de programación se le llama programación estructurada.



Programación Orientada a Objetos

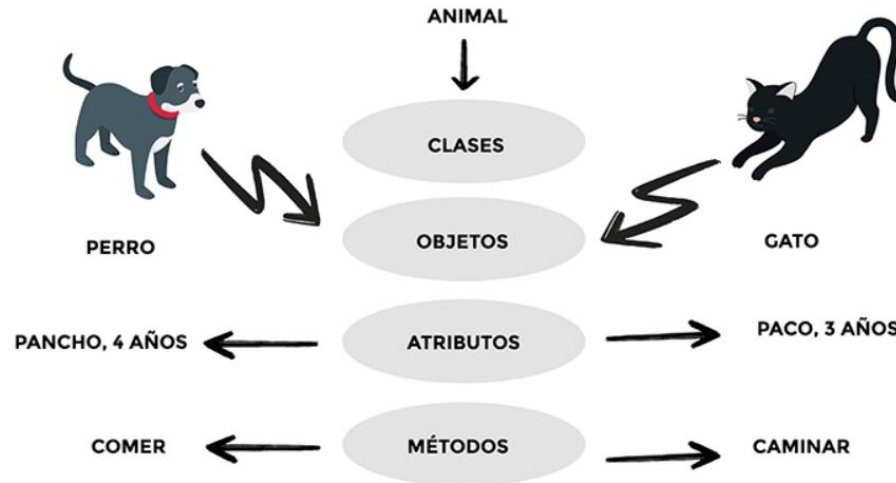
La orientación a objetos ha roto con esta forma de hacer las cosas. Con este nuevo paradigma el proceso se centra en simular los elementos de la realidad asociada al problema de la forma más cercana posible. La abstracción que permite representar estos elementos se denomina objeto, y tiene las siguientes características:

- Están formados por un conjunto de atributos, que son los datos que le caracterizan y un conjunto de operaciones que definen su comportamiento.
- Las operaciones asociadas a un objeto actúan sobre sus atributos para modificar su estado.
- Las aplicaciones orientadas a objetos están formadas por un conjunto de objetos que interaccionan entre sí por medio de las operaciones para producir resultados.
- Los objetos similares se abstraen en clases, se dice que un objeto es una instancia de una clase.

Programación Orientada a Objetos

Cuando se ejecuta un programa orientado a objetos ocurren tres sucesos:

1. Los objetos se crean a medida que se necesitan.
2. Los mensajes se mueven de un objeto a otro (o del usuario a un objeto) a medida que el programa procesa información o responde a la entrada del usuario.
3. Cuando los objetos ya no se necesitan, se borran y se libera la memoria.



Programación Orientada a Objetos

En este paradigma, los programas se modelan en torno a objetos que aglutinan toda la funcionalidad relacionada con ellos. De este modo en lugar de crear una serie de funciones sin conexión alguna entre ellas, en POO se crean clases, que representan entidades que quieres manejar en tu programa. Por ejemplo, facturas, líneas de factura, clientes, coches... o cualquier entidad que necesites gestionar conceptualmente. En ese sentido, la POO gira más en torno a los datos que en torno a la lógica.

Programación Orientada a Objetos

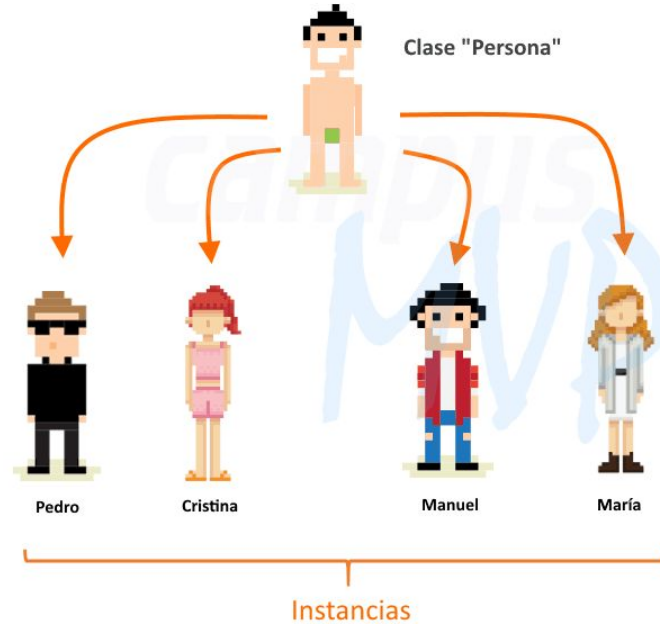
Una clase es una plantilla. Define de manera genérica cómo van a ser los objetos de determinado tipo. Por ejemplo, en un juego, una clase para representar a personas puede llamarse Persona y tener una serie de atributos como nombre, apellidos o edad, y una serie de comportamientos que pueden tener, como hablar(), caminar() o comer() y que se implementan como métodos de la clase.

Programación Orientada a Objetos

Una clase por sí sola no sirve de nada, pues no es más que un concepto, sin entidad real. Para poder utilizar una clase en un programa lo que hay que hacer es instanciarla. Instanciar una clase consiste en crear un nuevo objeto concreto de la misma. Es decir, un objeto es ya una entidad concreta que se crea a partir de la plantilla que es la clase. Este nuevo objeto tiene ya "existencia" real, puesto que ocupa memoria y se puede utilizar en el programa. Así un objeto puede ser una persona que se llama Cristina López, de 37 años y que en nuestro programa podría hablar, caminar o comer, que son los comportamientos que están definidos en la clase.

Programación Orientada a Objetos

Ejemplo



Programación Orientada a Objetos

De este modo, si tenemos que manejar personas podemos ir creándolas a medida que las necesitemos, y actuar sobre ellas individualmente. Cada una tiene sus propios datos y sus propias acciones.

La clase define de forma genérica cómo son las personas, y los objetos son personas concretas.

La POO es muy potente porque nos permite modelar de manera sencilla datos y comportamientos complejos del mundo real. Al poder manejar los datos y los comportamientos de cada objeto de manera independiente nos evita tener que mantener datos globales y coordinar todo eso.

Pilares de la P00

Para poder manejar de manera eficiente las clases y los objetos que se generan con la Programación Orientada a Objetos son necesarios algunos principios que ayudan a reducir la complejidad, ser más eficientes y evitar problemas. Son los 4 pilares de la P00. Todos los lenguajes orientados a objetos los implementan de una u otra manera, y es indispensable conocerlos bien. Estos son:

- Encapsulación
- Abstracción
- Herencia
- Polimorfismo

Encapsulación

La encapsulación es la característica de un lenguaje POO que permite que todo lo referente a un objeto quede aislado dentro de éste. Es decir, que todos los datos referentes a un objeto queden "encerrados" dentro de éste y sólo se puede acceder a ellos a través de los miembros que la clase proporcione (propiedades y métodos).

Por ejemplo, en el caso de las personas que estábamos viendo, toda la información sobre éstas (nombre, apellidos, edad... y cualquier otro dato interno que se utilice y que no necesariamente se ve desde el exterior del objeto) está circunscrito al ámbito de dicha persona.

Abstracción

La clase debe representar las características de la entidad hacia el mundo exterior, pero ocultando la complejidad que llevan aparejada. O sea, nos abstrae de la complejidad que haya dentro dándonos una serie de atributos y comportamientos (propiedades y funciones) que podemos usar sin preocuparnos de qué pasa por dentro cuando lo hagamos.

Así, una clase (y por lo tanto todos los objetos que se crean a partir de ella) debe exponer para su uso solo lo que sea necesario. Cómo se haga "por dentro" es irrelevante para los programas que hagan uso de los objetos de esa clase.

Abstracción

En el ejemplo de las personas podría ser la acción `hablar()`. Ésta puede suponer que se genere una voz sintética a partir de un texto que se le indica como parámetro de la acción para lo cual quizá ocurran un montón de cosas: se llama a un componente para síntesis de voz que está en la nube, se lanza la síntesis de voz en el dispositivo local de audio y se anota en una base de datos la frase que se ha pronunciado para guardar un histórico entre otras cosas. Pero todo esto es indiferente para el programa que hace uso de esta funcionalidad. El programa simplemente tiene acceso a un objeto `Cristina` y llama a la función `hablar()`. No tiene ni idea de toda la complejidad interna que puede suponer. Si mañana cambiamos el modo de sintetizar la voz o cualquier otra acción interna, es indiferente para el programa que usa nuestros objetos de tipo `Persona`.

Herencia

Desde el punto de vista de la genética, cuando una persona obtiene de sus padres ciertos rasgos (el color de los ojos o de la piel, una enfermedad genética, etc...) se dice que los hereda. Del mismo modo en POO cuando una clase hereda de otra obtiene todos los rasgos que tuviese la primera.

Dado que una clase es un patrón que define cómo es y cómo se comporta una cierta entidad, una clase que hereda de otra obtiene todos los rasgos de la primera y añade otros nuevos y además también puede modificar algunos de los que ha heredado.

A la clase de la que se hereda se le llama clase base o padre, y a la clase que hereda de ésta se le llama clase derivada o hija.

Herencia

Así, en nuestro juego que involucra personas, podemos tener clases de personas más especializadas para representar personajes especiales del juego. Por ejemplo, podríamos definir clases como Pirata, Piloto o Estratega que heredan de la clase Persona. Todos los objetos de estas clases heredan las propiedades y los métodos de Persona, pero pueden particularizar algunos de ellos y además añadir cosas propias.

Por ejemplo, los objetos de la clase Pirata tienen un método nuevo que es Abordar() que en el juego sirve para asaltar un barco enemigo. Pero además presentan una propiedad que solo tienen los piratas llamada Sobrenombre, que es el nombre por el que se les conoce (un pirata puede ser de nombre Hızır pero su sobrenombre es Barbaroja).

Polimorfismo

En POO, el concepto de polimorfismo se refiere al hecho de que varios objetos de diferentes clases, pero con una base común, se pueden usar de manera indistinta, sin tener que saber de qué clase exacta son para poder hacerlo.

Supongamos que en nuestro juego tenemos un montón de personajes que están juntos en un mismo escenario. Hay varios piratas, algunos estrategas y un montón de otros tipos de personas. En un momento dado necesitamos que todos se pongan a hablar. Cada uno lo hace de una forma diferente, ya que son tipos de personajes distintos. Sería algo bastante tedioso tener que localizar primero a los de un tipo y hacerlos hablar, luego a los de otro y así sucesivamente. La idea es que puedas tratarlos a todos como personas, independientemente del tipo específico de persona que sean y simplemente decirles que hablen.

Polimorfismo

Al derivar todos de la clase Persona todos pueden hablar, y al llamar al método Hablar() de cada uno de ellos se utilizará el proceso adecuado según el tipo (los piratas meterán su expresión "Arrrg", los pilotos dirán "Entrando en pista", y los estrategas añadirán a todo "Déjame que lo piense bien"). Todo esto de manera transparente para el programador. Esto es el polimorfismo.

De hecho, el polimorfismo puede ser más complicado que eso ya que se puede dar también mediante la sobrecarga de métodos y, sobre todo, a través del uso de interfaces.

El polimorfismo nos permite utilizar a los objetos de manera genérica, aunque internamente se comporten según su variedad específica.

Ciclo de vida de los objetos

- Creación:
 - Reserva de memoria => `Persona persona = new Persona(...)`
 - Inicialización de atributos => Constructores
- Manipulación:
 - Acceso a atributos => `persona.nombre`
 - Invocación de métodos => `persona.comer()`
- Destrucción:
 - Al liberar la memoria => Garbage collector

Niveles de accesibilidad

Modificador	La misma clase	Mismo paquete	Subclase	Otro paquete
private	Sí	No	No	No
default	Sí	Sí	No	No
protected	Sí	Sí	Sí/No	No
public	Sí	Sí	Sí	Sí

Niveles de accesibilidad

- Clases: public, protected y private
- Atributos: private, protected
- Constructores: public, protected y private
- Métodos: public, protected y private

Lenguaje de Modelado Unificado

UML (Unified Modeling Language) es un lenguaje estándar usado en ingeniería de software para representar visualmente sistemas y procesos mediante diagramas. Permite a los desarrolladores crear modelos visuales que describen la estructura y el comportamiento de un sistema, facilitando la comprensión, el diseño y la documentación de software.

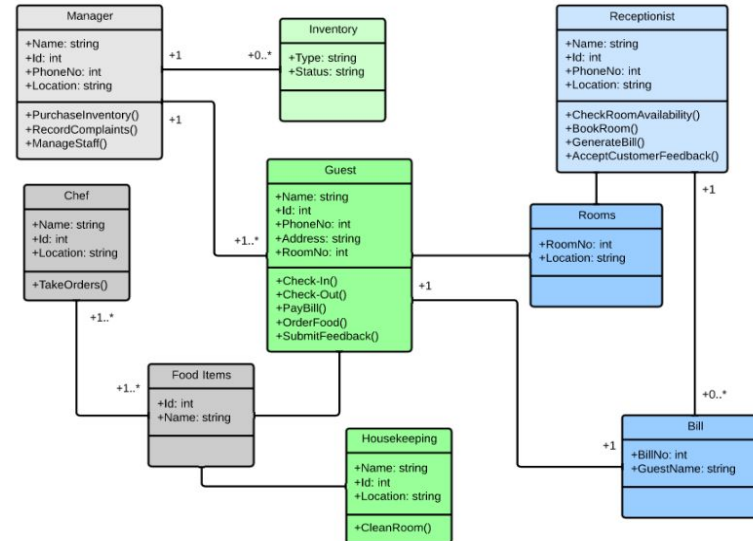
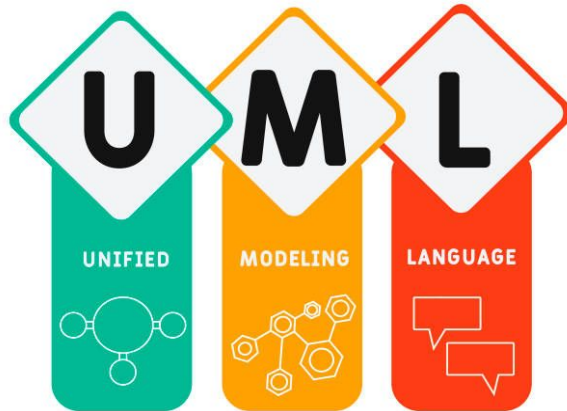


Diagrama de clases

Un diagrama de clases en Lenguaje Unificado de Modelado (UML) es un tipo de diagrama que describe la estructura de un sistema (abstracción de un modelo real) mostrando las clases del sistema, sus atributos, operaciones (o métodos), y las relaciones entre los objetos. Los diagramas de clase muestran lo que el sistema puede hacer (análisis) y el cómo puede ser construido (diseño).

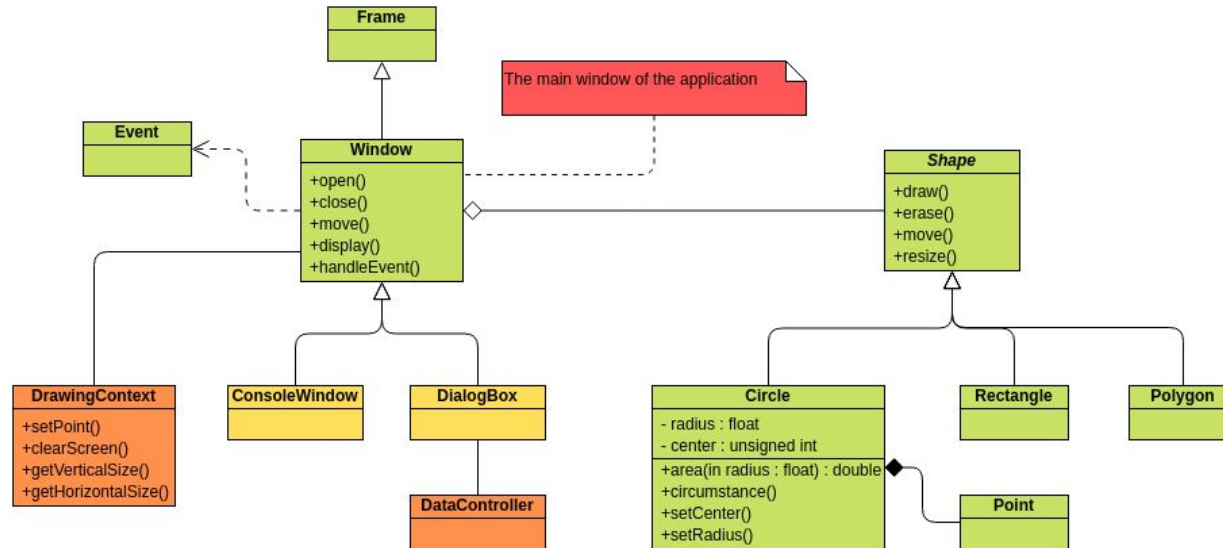


Diagrama de clases

Los diagramas de clases ofrecen una serie de beneficios para toda organización. Usa los diagramas de clases UML para:

- Ilustrar modelos de datos para sistemas de información, sin importar qué tan simples o complejos sean.
- Comprender mejor la visión general de los esquemas de una aplicación.
- Expresar visualmente cualquiera necesidad específica de un sistema y divulgar esa información en toda la empresa.
- Crear diagramas detallados que resalten cualquier código específico que será necesario programar e implementar en la estructura descrita.
- Ofrecer una descripción independiente de la implementación sobre los tipos empleados en un sistema que son posteriormente transferidos entre sus componentes.

Diagrama de clases

Hay que identificar:

- **Nombres:** Clases y sus atributos
- **Adjetivos:** Valores de las atributos
- **Verbos:** Comportamiento de las clases (métodos)

“El **coche** tiene **color rojo** y se **mueve**”

“El **documento** tiene **letra grande** y se **muestra**”

Procedimiento:

1. Identificar clases
2. Identificar y depurar relaciones
3. Identificar atributos de clases y relaciones
4. Añadir herencia
5. Comprobar casos de uso (iterar)
6. Añadir y simplificar métodos

Diagrama de clases

Representación de Clase

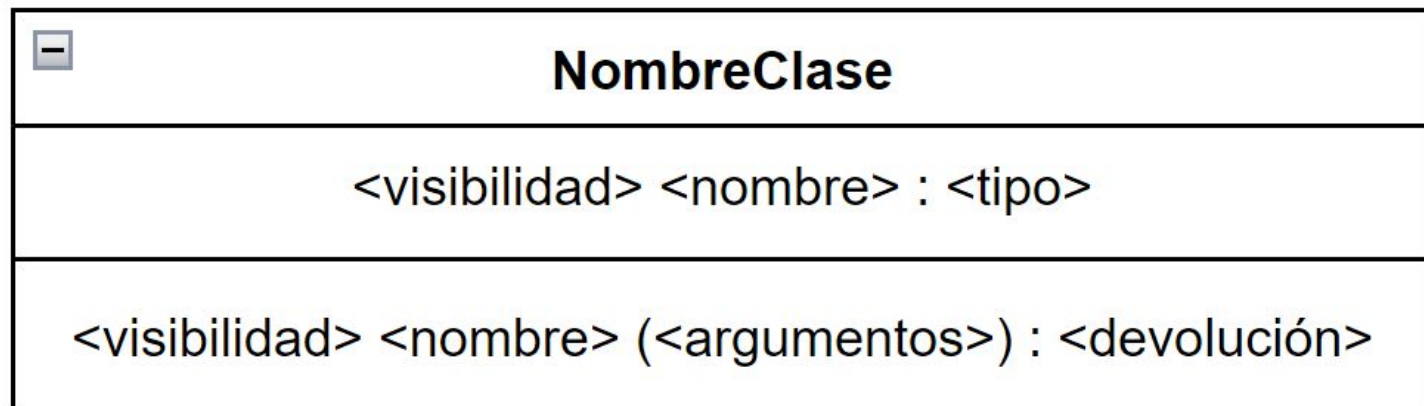
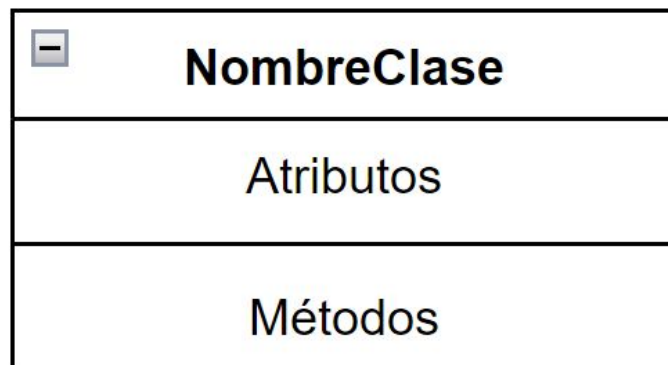


Diagrama de clases

Tanto los atributos como las funciones incluyen al principio de su descripción la visibilidad que tendrá. Esta visibilidad se identifica escribiendo un símbolo y podrá ser:

(+) Pública. Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.

(-) Privada. Representa que se puede acceder al atributo o función únicamente desde la misma clase.

(#) Protegida. Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas).

Diagrama de clases

Visibilidad

	NombreClase
public	+ atributoPublico
protected	# atributoProtegido
private	- atributoPrivado
static	<u>atributoDeClase</u>
public	+ metodoPublico
protected	# metodoProtegido
private	- metodoPrivado
static	<u>metodoDeClase</u>

Diagrama de clases

Ejemplo clase

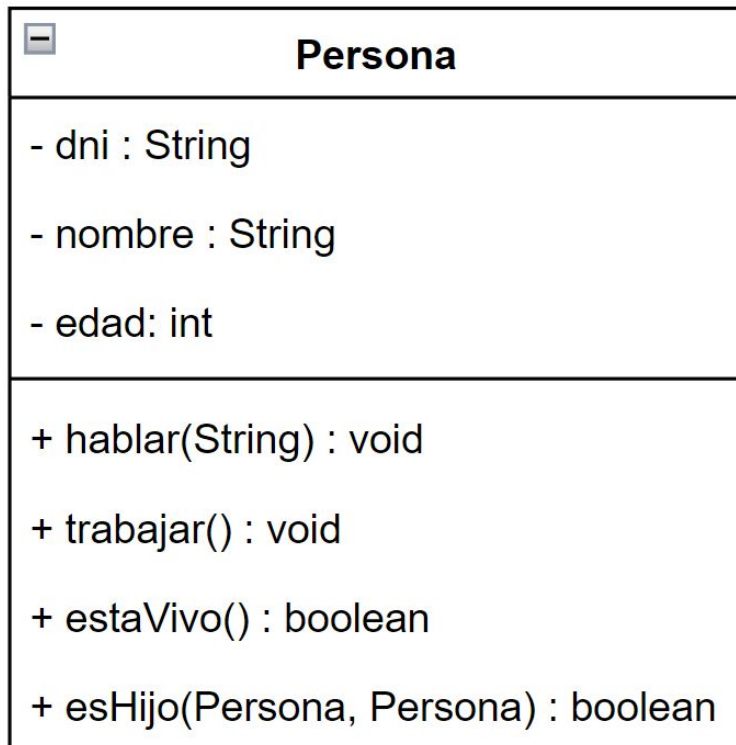


Diagrama de clases

Representación clase abstracta

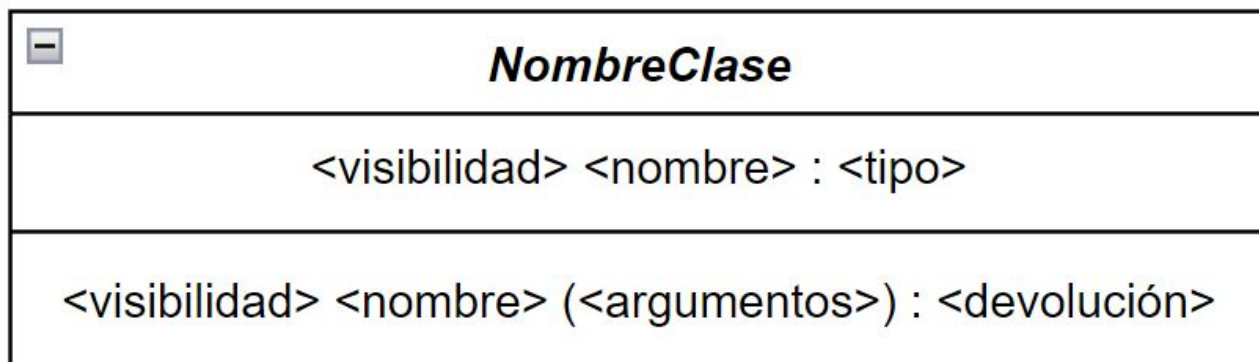
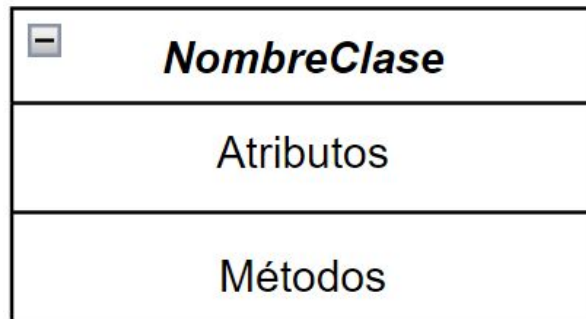


Diagrama de clases

Ejemplo clase abstracta

Método abstracto

Método abstracto

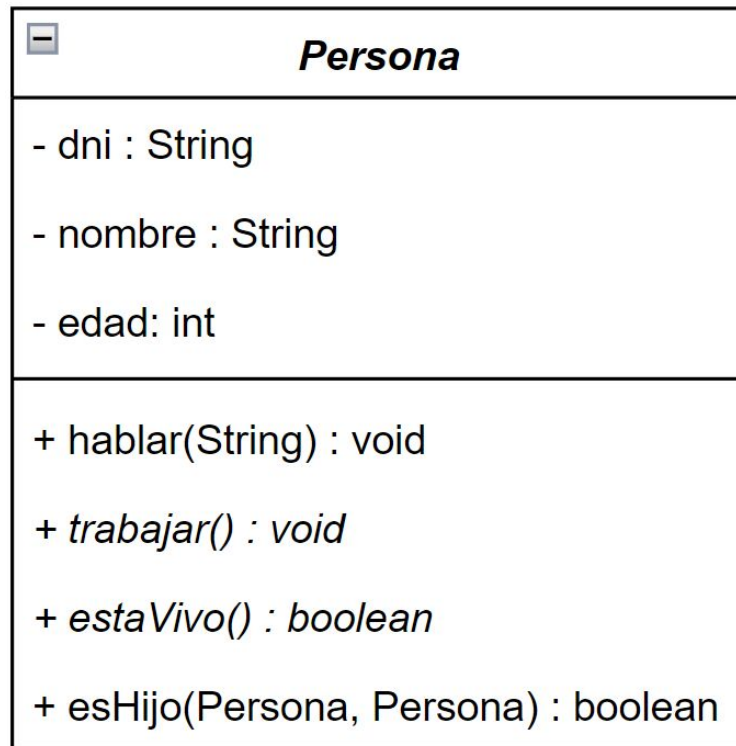


Diagrama de clases

Representación clase genérica

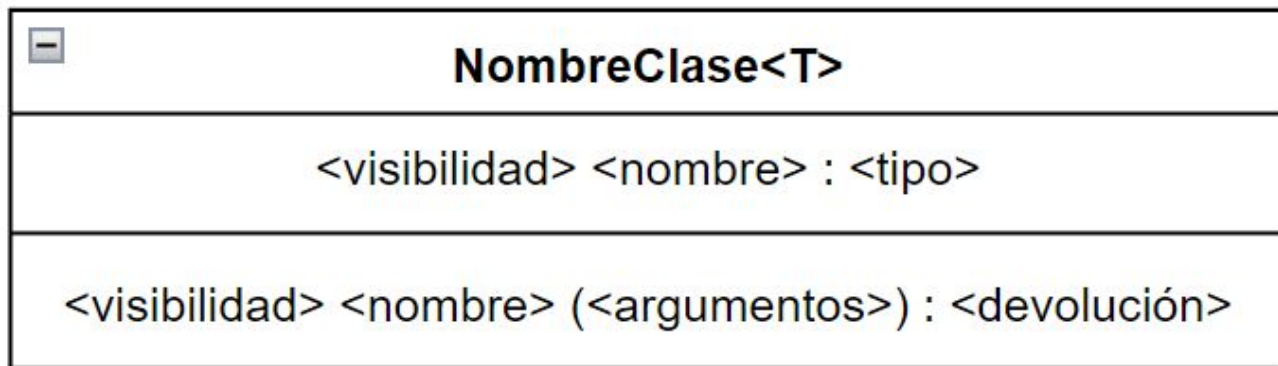
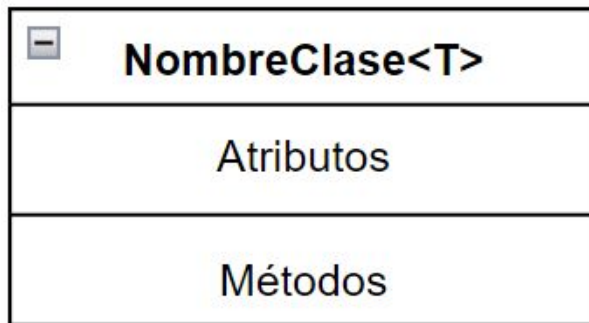


Diagrama de clases

Ejemplo clase genérica

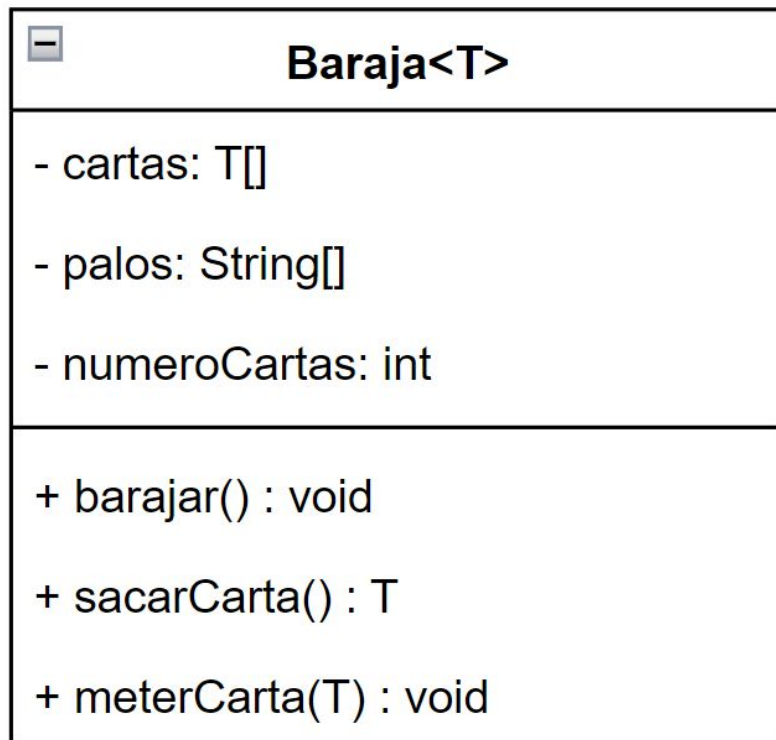


Diagrama de clases

Representación de Interfaz

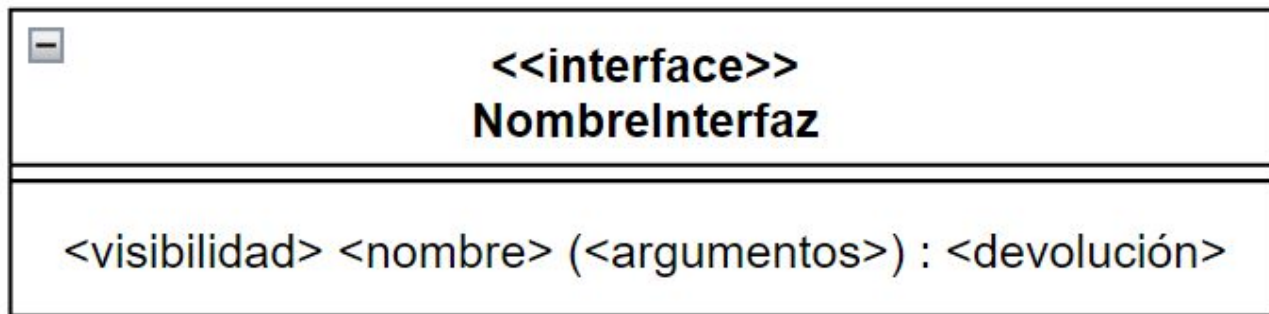
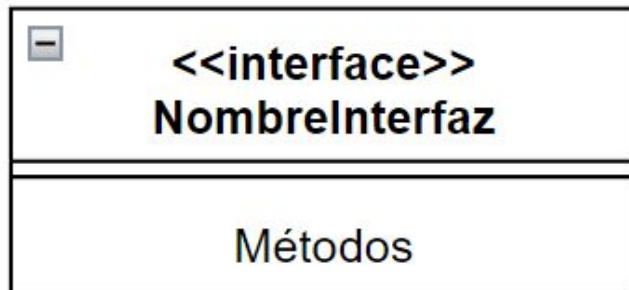


Diagrama de clases

Ejemplo interfaz

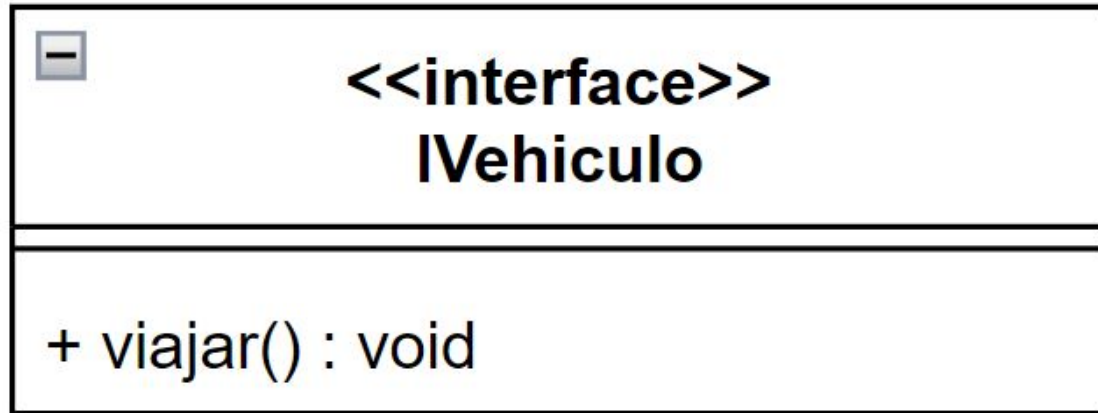


Diagrama de clases

Representación de Enumerado

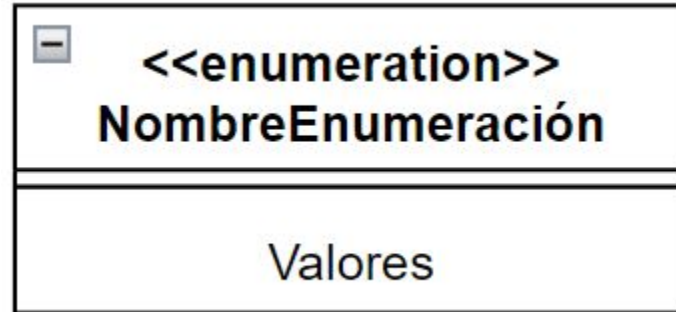


Diagrama de clases

Ejemplo de Enumerado

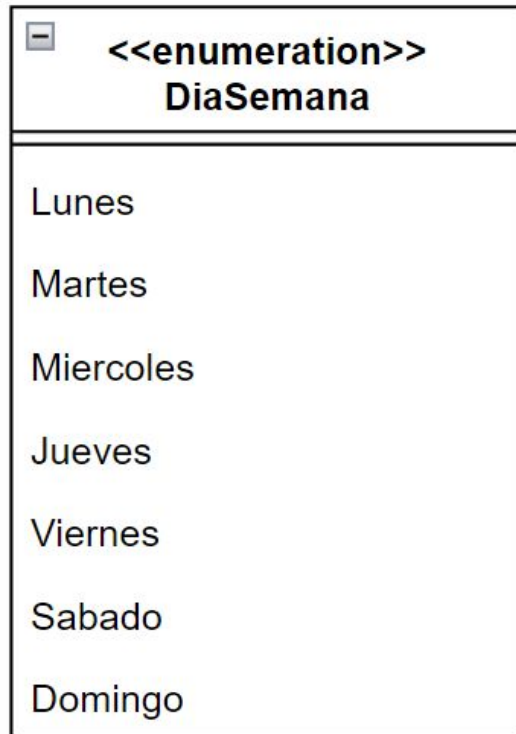


Diagrama de clases

Tener en cuenta:

- Los atributos no deben ser objetos (usar relaciones en tal caso)

Los diagramas de clases no suelen incluir (son detalles de implementación):

- Constructores
- Métodos/Propiedades de acceso (getters y setters)

Diagrama de clases

Relaciones

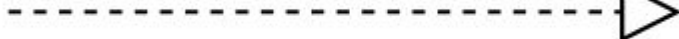
Tipo	Notación
Asociación	
Herencia	
Implementación	

Diagrama de clases

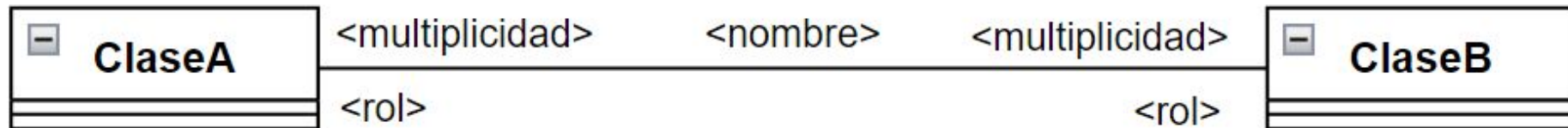
Asociación

Es una relación estructural que describe un conjunto de enlaces, los cuales son conexiones entre objetos. La asociación se representa con una línea continua, posiblemente dirigida, que a veces incluye una etiqueta. A menudo se incluyen otros elementos para indicar la multiplicidad y roles de los objetos involucrados.

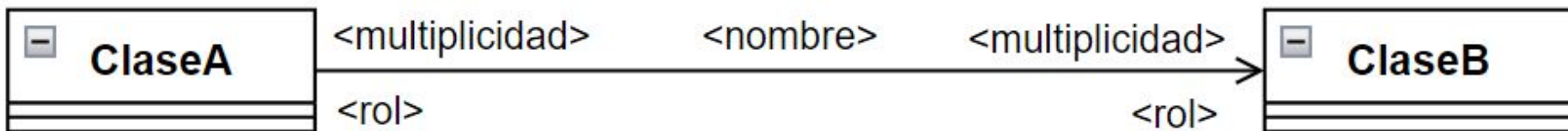
Dada una asociación entre dos clases, se puede navegar desde un objeto de una de ellas hasta uno de los objetos de la otra, y viceversa. Es posible que la asociación se dé de manera recursiva en un objeto, esto significa que dado un objeto de la clase se puede conectar con otros objetos de la misma clase. Las relaciones de asociaciones se utilizan cuando se quieren representar relaciones estructurales.

Diagrama de clases

Asociación no dirigida



Asociación dirigida



Para indicar el papel que juega una clase en una asociación se puede especificar un nombre de rol. Se representa en el extremo de la asociación junto a la clase que desempeña dicho rol.

Diagrama de clases

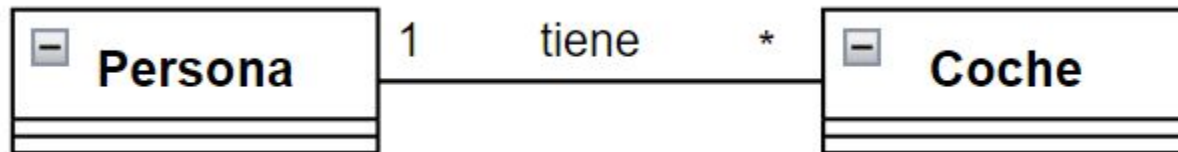
Multiplicidades

Multiplicidad	Significado
1	Uno y solamente uno
0..1	Cero o uno
N..M	Desde N hasta M
*	Cero o varios
0..*	Cero o varios
1..*	Uno o varios (al menos uno)
N..*	N o varios (al menos N)

<= Por defecto

Diagrama de clases

Ejemplo asociación



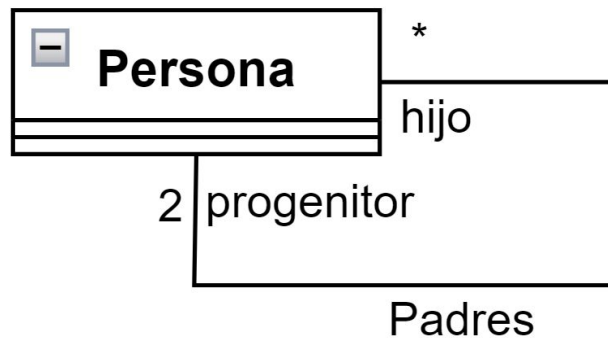
```
class Persona
{
    private Coche[] coches;
}
```

4 referencias

```
class Coche
{
    // No puede ser nulo porque la multiplicidad es 1
    private Persona propietario;
}
```

Diagrama de clases

Ejemplo asociación reflexiva



Traducido directamente

```
class Persona
{
    private Persona[] hijos;

    // Su tamaño tiene que ser de 2
    private Persona[] progenitores;
}
```

Posible adaptación para facilitar la implementación

```
class Persona
{
    private Persona[] hijos;
    private Persona padre;
    private Persona madre;
}
```

Diagrama de clases

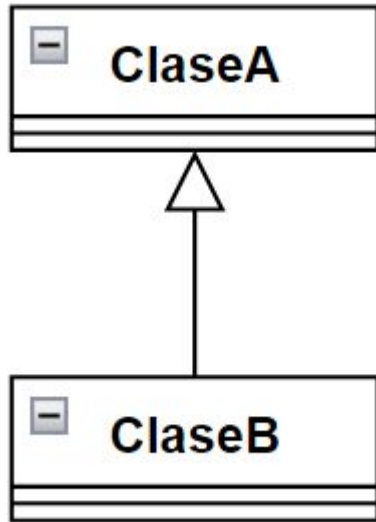
Herencia

Uno de las características principales de la orientación a objetos es poder asociar a los objetos (clases) con características y operaciones que puedan heredarse entre ellos, siempre y cuando estos estén ligados a un entorno en común. La orientación a objetos se refiere a esto como herencia y en UML se denomina generalización. En una generalización no hay multiplicidad ni roles.

Una herencia múltiple se presenta cuando una clase subordinada proviene de varias clases principales. En C#/Java no se permite la herencia múltiple.

Diagrama de clases

Herencia simple



Herencia múltiple

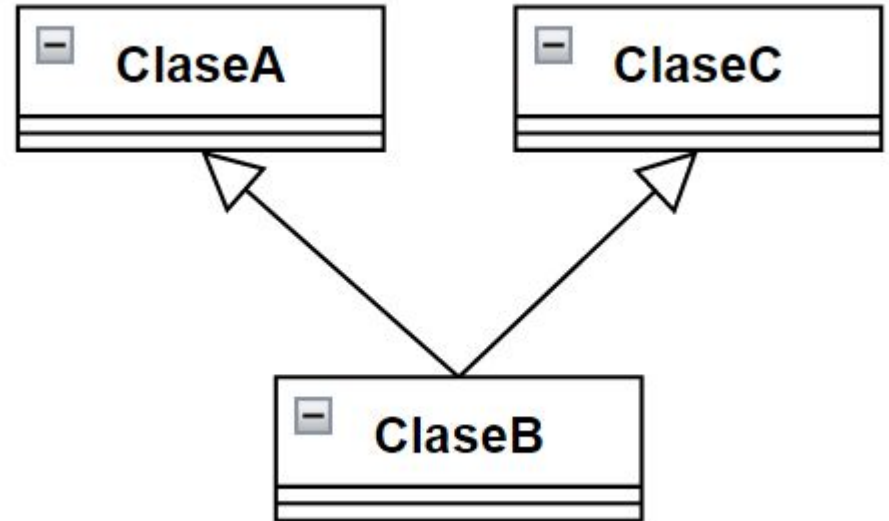
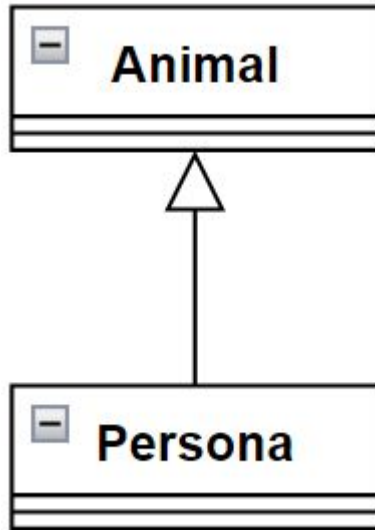


Diagrama de clases

Herencia simple



Herencia múltiple

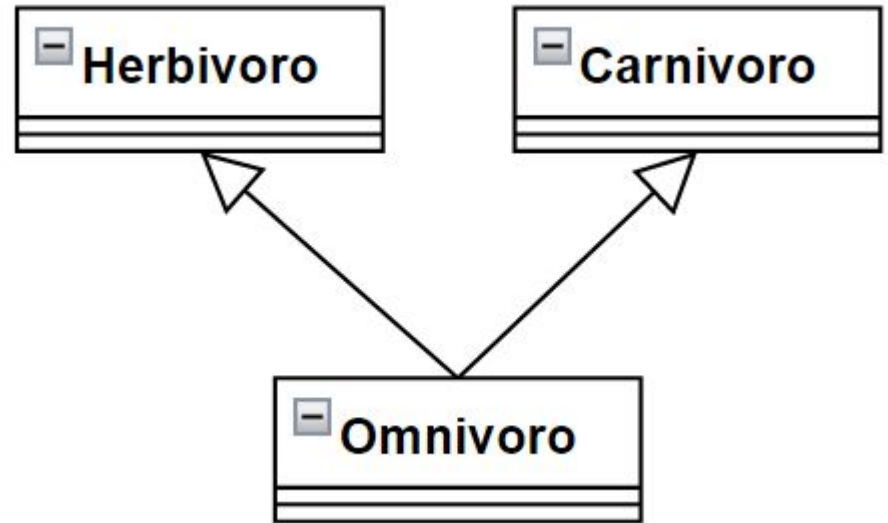
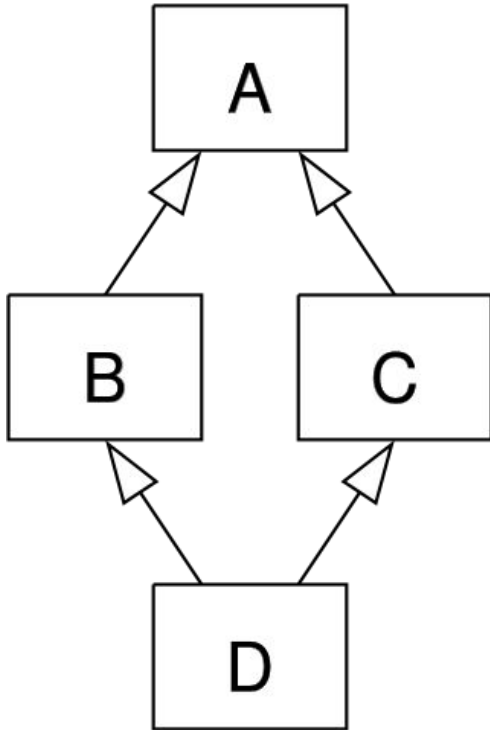


Diagrama de clases

Problema herencia múltiple



En los lenguajes de programación orientada a objetos, el problema del diamante es una ambigüedad que surge cuando dos clases B y C heredan de A, y la clase D hereda de B y C. Si un método en D llama a un método definido en A, ¿qué clase lo hereda, B o C?

Lenguajes que sólo permiten herencia simple (como C# y Java) permiten la implementación múltiple de interfaces. Las interfaces son esencialmente clases base abstractas con todos los métodos abstractos y sin miembros de datos. El problema es por tanto evitado ya que siempre hay sólo una implementación para un método o propiedad específico y no surge ninguna ambigüedad.

Diagrama de clases

Problema herencia múltiple

Para diseñar especialización múltiple en C#/Java:

- se hereda de una clase y se asocia con el resto (asociación dirigida)
- puede ser necesario reorganizar la jerarquía de clases

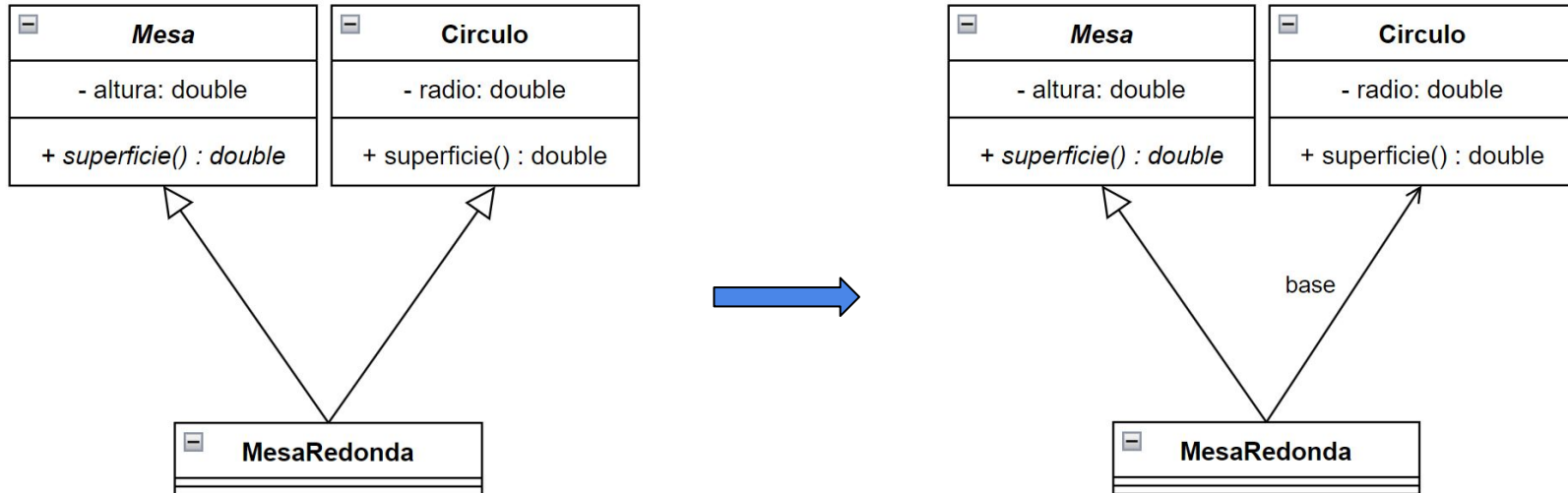
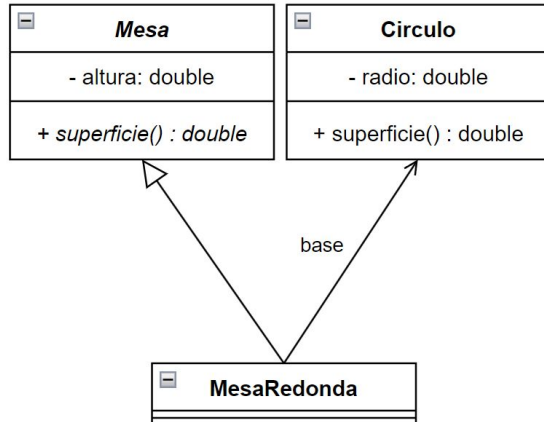


Diagrama de clases

Problema herencia múltiple

```
public class Circulo {  
    private double radio;  
  
    public double superficie() {  
        return Math.PI * Math.pow(radio, 2);  
    }  
}
```

```
public abstract class Mesa {  
    private double altura;  
  
    public double getAltura() {  
        return altura;  
    }  
  
    public abstract double superficie();  
}
```



```
public class MesaRedonda extends Mesa {  
    private Circulo base;  
  
    public double superficie() {  
        return base.superficie() * getAltura();  
    }  
}
```

Diagrama de clases

Implementación

Las interfaces son mecanismos que permiten a los lenguajes orientados a objetos con herencia lineal, como C# o Java, acceder a la semántica de los lenguajes orientados a objetos con herencia múltiple, como C++. Sin embargo, las posibilidades de las interfaces difieren de aquellas que aporta la herencia múltiple.

La idea es que las interfaces son como contratos para las clases que las obligan a definir los métodos establecidos en cláusulas de funcionalidad.

Las clases que se comprometen y cumplen con estos contratos tienen implementada esa funcionalidad y por tanto son capaces de llevarla a cabo.

Diagrama de clases

Ejemplo Implementación

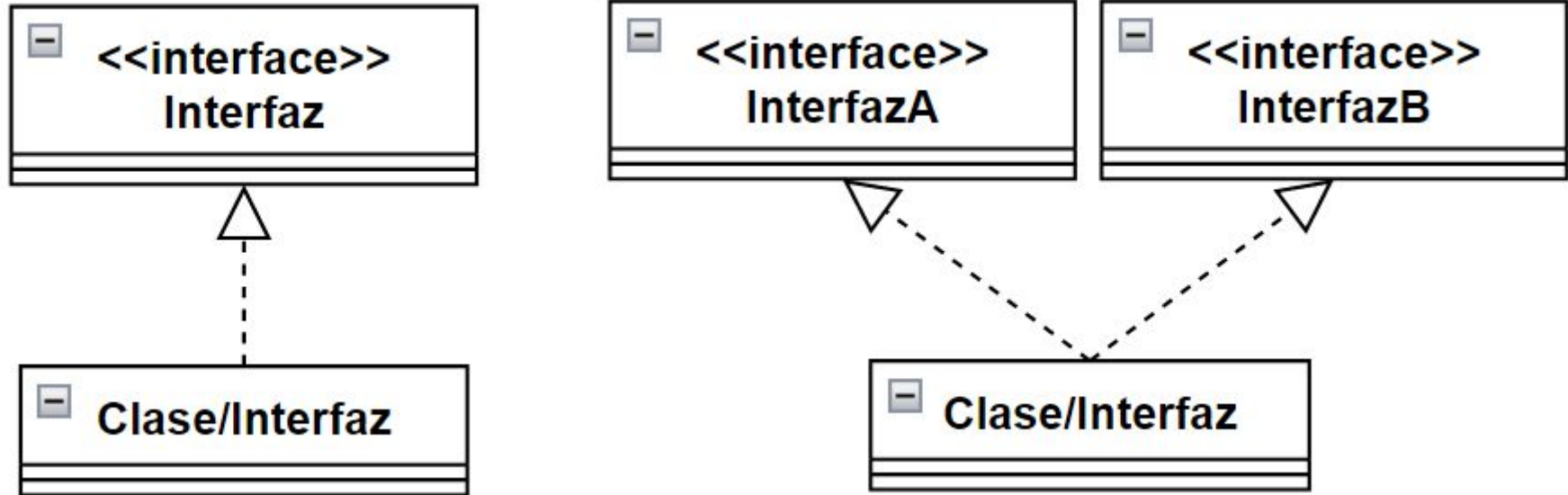


Diagrama de clases

Ejemplo Implementación

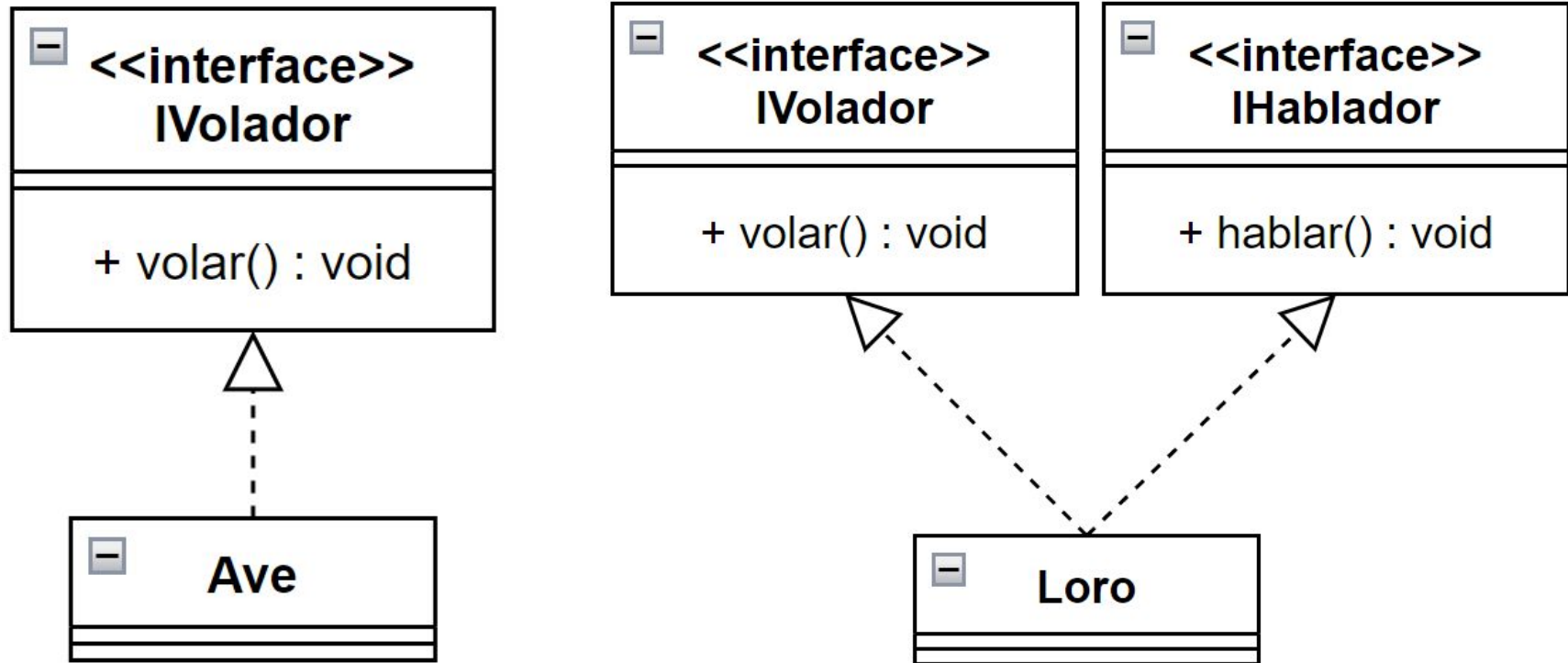


Diagrama de clases

Ejemplo completo diagrama de clases

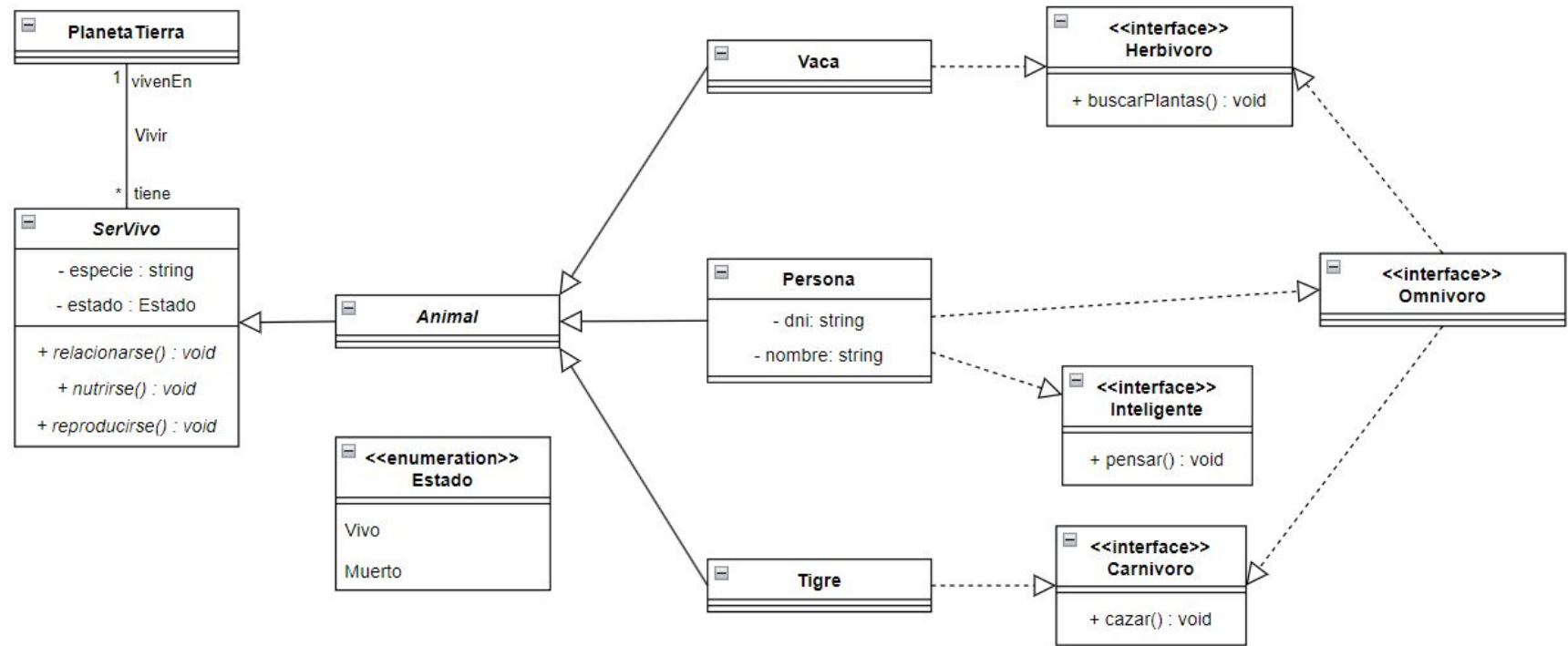


Diagrama de clases

Ejemplo completo diagrama de clases

```
public abstract class SerVivo {  
  
    public enum Estado { Vivo, Muerto };  
  
    private String especie;  
    private Estado estado;  
    // Según el diagrama se tendría  
    // que llamar 'viveEn', como no hay confusión  
    // se podría obviar y llamarlo así  
    private PlanetaTierra planeta;  
  
    public abstract void relacionarse();  
    public abstract void nutrirse();  
    public abstract void reproducirse();  
}
```

```
public class PlanetaTierra {  
    // Según el diagrama se tendría  
    // que llamar 'tiene', como no hay confusión  
    // se podría obviar y llamarlo así  
    private SerVivo[] seresVivos;  
}
```

Diagrama de clases

Ejemplo completo diagrama de clases

```
public class Animal extends SerVivo {  
  
    @Override  
    public void relacionarse() {  
  
    }  
  
    @Override  
    public void nutrirse() {  
  
    }  
  
    @Override  
    public void reproducirse() {  
  
    }  
  
}
```

```
public interface Herbivoro {  
  
    void buscarPlantas();  
  
}  
  
public interface Carnivoro {  
  
    void cazar();  
  
}  
  
public interface Omnivoro extends Herbivoro, Carnivoro {  
  
}  
  
public interface Inteligente {  
  
    void pensar();  
  
}
```


Diagrama de clases

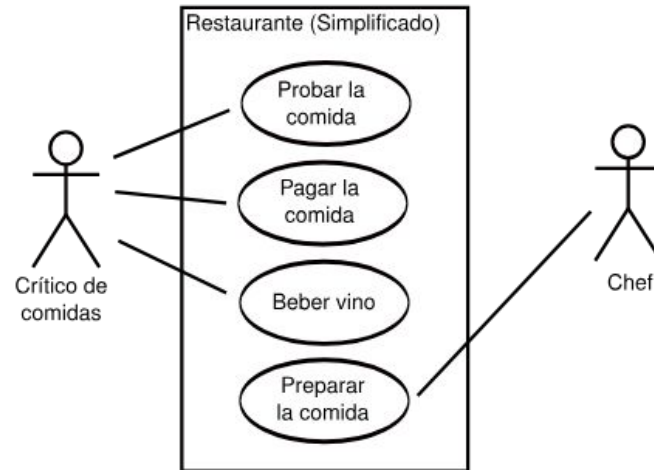
Ejemplo completo diagrama de clases

```
public class Persona extends Animal implements Omnivoro, Inteligente {  
    private String dni;  
    private String nombre;  
  
    @Override  
    public void buscarPlantas() {  
  
    }  
  
    @Override  
    public void cazar() {  
  
    }  
  
    @Override  
    public void pensar() {  
  
    }  
}
```

```
public class Vaca extends Animal implements Herbivoro {  
  
    @Override  
    public void buscarPlantas() {  
  
    }  
}  
  
public class Tigre extends Animal implements Carnivoro {  
  
    @Override  
    public void cazar() {  
  
    }  
}
```

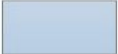
Diagrama de casos de uso

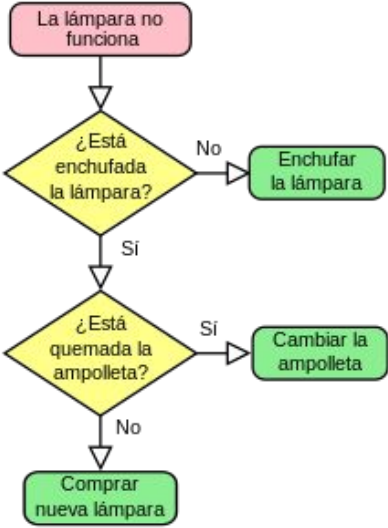
Los diagramas de casos de uso en UML representan las interacciones entre un sistema y sus actores (usuarios o entidades externas) mostrando cómo se usará el sistema en diferentes situaciones. Estos diagramas describen las funciones o acciones que los usuarios pueden realizar en un sistema y cómo se relacionan con él, ayudando a comprender los requisitos funcionales y las relaciones entre los actores y el sistema en un nivel de alto nivel.



Diagramas de flujo

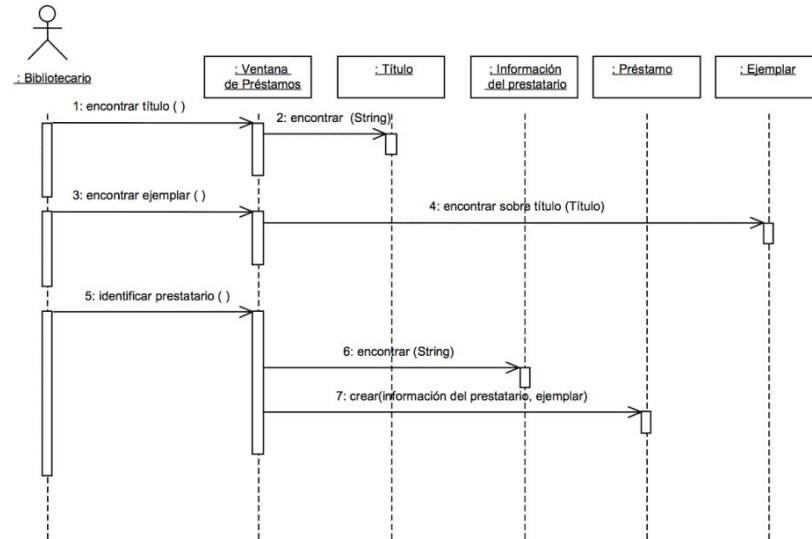
Los diagramas de flujo en el desarrollo de software son representaciones visuales que ilustran la secuencia de pasos lógicos y decisiones en un algoritmo, proceso o sistema de software. Se utilizan para diseñar, documentar y comprender la lógica detrás de un procedimiento o función en un sistema informático. Estos diagramas consisten en símbolos gráficos conectados por flechas que muestran la secuencia de ejecución.

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Linea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso



Diagramas de secuencia

Los diagramas de flujo en el desarrollo de software son representaciones visuales que ilustran la secuencia de pasos lógicos y decisiones en un algoritmo, proceso o sistema de software. Se utilizan para diseñar, documentar y comprender la lógica detrás de un procedimiento o función en un sistema informático. Estos diagramas consisten en símbolos gráficos conectados por flechas que muestran la secuencia de ejecución.



Diagramas de estado

Los diagramas de estado son una representación visual en el desarrollo de software que describe el comportamiento de un objeto a lo largo de su ciclo de vida, mostrando los diferentes estados en los que puede existir y las transiciones entre esos estados.

